

Assessment-3 (PART-1)

Girija G

Experiment-8(a) Construct Predictive parse table using C language.

Hint: Consider the input grammar without left recursion, find FIRST and FOLLOW for each non-terminal and then construct the parse table.

ALGORITHM

1. Read the number of productions and the grammar rules from the user.
2. Remove immediate left recursion from the given grammar and generate a new grammar.
3. Collect all non-terminals from the transformed grammar.
4. Identify all terminals present in the grammar and add \$ as the end marker.
5. Initialize FIRST and FOLLOW sets as empty for every non-terminal.
6. Compute FIRST sets iteratively until no further changes occur.
7. Initialize FOLLOW of the start symbol with \$.
8. Compute FOLLOW sets iteratively using FIRST rules until stable.
9. Construct the LL(1) parsing table using FIRST and FOLLOW sets.
10. Display the transformed grammar, FIRST sets, FOLLOW sets, and parsing table.

SOURCE CODE

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 50
#define NTMAX 20

char prod[MAX][MAX], g[MAX][MAX], NT[NTMAX], TR[MAX];
char FIRST[NTMAX][MAX], FOLLOW[NTMAX][MAX], table[NTMAX][MAX][MAX];
int nProd, nG = 0, nNT = 0, nT = 0;

/* ----- Utility Functions ----- */
int isNT(char c) {
    int i;
    for (i = 0; i < nNT; i++)
        if (NT[i] == c) return 1;
    return 0;
}

int ntIndex(char c) {
    int i;
    for (i = 0; i < nNT; i++)
        if (NT[i] == c) return i;
    return -1;
}
```

```

}

int tIndex(char c) {
    int i;
    for (i = 0; i < nT; i++)
        if (TR[i] == c) return i;
    return -1;
}

void add(char *set, char c) {
    int l;
    if (c == '\0') return;
    if (!strchr(set, c)) {
        l = strlen(set);
        set[l] = c;
        set[l + 1] = '\0';
    }
}

/* ----- Left Recursion Removal ----- */
void removeLR() {
    int used[256] = {0};
    int i, j, a, b;
    char A, Ap;
    char alpha[10][MAX], beta[10][MAX];

    for (i = 0; i < nProd; i++) {
        A = prod[i][0];
        if (used[(int)A]) continue;
        used[(int)A] = 1;

        a = b = 0;

        for (j = 0; j < nProd; j++) {
            if (prod[j][0] == A) {
                if (prod[j][3] == A)
                    strcpy(alpha[a++], prod[j] + 4);
                else
                    strcpy(beta[b++], prod[j] + 3);
            }
        }

        if (a > 0) {
            Ap = tolower(A);
            NT[nNT++] = A;
            NT[nNT++] = Ap;

            for (j = 0; j < b; j++)
                sprintf(g[nG++], "%c->%s%c", A, beta[j], Ap);
            for (j = 0; j < a; j++)

```

```

        sprintf(g[nG++], "%c->%s%c", Ap, alpha[j], Ap);

        sprintf(g[nG++], "%c->#", Ap);
    } else {
        NT[nNT++] = A;
        for (j = 0; j < b; j++)
            sprintf(g[nG++], "%c->%s", A, beta[j]);
    }
}
}

/* ----- Find Terminals ----- */
void findTerminals() {
    int i, j;
    for (i = 0; i < nG; i++)
        for (j = 3; g[i][j]; j++)
            if (!isNT(g[i][j]) && g[i][j] != '#' && !strchr(TR, g[i][j]))
                add(TR, g[i][j]);

    add(TR, '$');
    nT = strlen(TR);
}

/* ----- FIRST & FOLLOW ----- */
void firstOfString(char *s, char *out) {
    int i, idx, j;
    if (s[0] == '#') {
        add(out, '#');
        return;
    }

    for (i = 0; s[i]; i++) {
        if (!isNT(s[i])) {
            add(out, s[i]);
            return;
        }
        idx = ntIndex(s[i]);
        for (j = 0; FIRST[idx][j]; j++)
            if (FIRST[idx][j] != '#')
                add(out, FIRST[idx][j]);

        if (!strchr(FIRST[idx], '#'))
            return;
    }
    add(out, '#');
}

void computeFIRST() {
    int changed = 1;
    int i, A, j;

```

```

char temp[MAX];

while (changed) {
    changed = 0;
    for (i = 0; i < nG; i++) {
        A = ntIndex(g[i][0]);
        temp[0] = '\0';
        firstOfString(g[i] + 3, temp);
        for (j = 0; temp[j]; j++)
            if (!strchr(FIRST[A], temp[j])) {
                add(FIRST[A], temp[j]);
                changed = 1;
            }
    }
}

void computeFOLLOW() {
    int changed = 1;
    int i, j, k, Aidx, Bidx;
    char *rhs, temp[MAX];

    add(FOLLOW[0], '$');

    while (changed) {
        changed = 0;
        for (i = 0; i < nG; i++) {
            Aidx = ntIndex(g[i][0]);
            rhs = g[i] + 3;

            for (j = 0; rhs[j]; j++) {
                if (isNT(rhs[j])) {
                    Bidx = ntIndex(rhs[j]);
                    temp[0] = '\0';
                    firstOfString(rhs + j + 1, temp);

                    for (k = 0; temp[k]; k++)
                        if (temp[k] != '#' && !strchr(FOLLOW[Bidx], temp[k])) {
                            add(FOLLOW[Bidx], temp[k]);
                            changed = 1;
                        }

                    if (!rhs[j + 1] || strchr(temp, '#'))
                        for (k = 0; FOLLOW[Aidx][k]; k++)
                            if (!strchr(FOLLOW[Bidx], FOLLOW[Aidx][k])) {
                                add(FOLLOW[Bidx], FOLLOW[Aidx][k]);
                                changed = 1;
                            }
                }
            }
        }
    }
}

```

```

    }
}

/* ----- Parsing Table ----- */
void computeTable() {
    int i, j, k, Aidx;
    char temp[MAX];

    for (i = 0; i < nG; i++) {
        Aidx = ntIndex(g[i][0]);
        temp[0] = '\0';
        firstOfString(g[i] + 3, temp);

        for (j = 0; temp[j]; j++) {
            if (temp[j] != '#')
                strcpy(table[Aidx][tIndex(temp[j])], g[i]);
            else
                for (k = 0; FOLLOW[Aidx][k]; k++)
                    strcpy(table[Aidx][tIndex(FOLLOW[Aidx][k])], g[i]);
        }
    }
}

```

```

int main() {
    int i, j;

    printf("Enter number of productions: ");
    scanf("%d", &nProd);

    printf("Enter productions:\n");
    for (i = 0; i < nProd; i++)
        scanf("%s", prod[i]);

    removeLR();
    findTerminals();
    computeFIRST();
    computeFOLLOW();
    computeTable();

    printf("\nGrammar After LR Removal:\n");
    for (i = 0; i < nG; i++)
        printf("%s\n", g[i]);

    printf("\nNT\tFIRST\tFOLLOW\n");
    for (i = 0; i < nNT; i++)
        printf("%c\t%s\t%s\n", NT[i], FIRST[i], FOLLOW[i]);

    printf("\nParsing Table:\n\t");
    for (i = 0; i < nT; i++)

```

```

    printf("%c\t", TR[i]);
    printf("\n");

    for (i = 0; i < nNT; i++) {
        printf("%c\t", NT[i]);
        for (j = 0; j < nT; j++)
            printf("%s\t", strlen(table[i][j]) ? table[i][j] : "-");
        printf("\n");
    }
    return 0;
}

```

INPUT

```

Enter number of productions: 5
Enter productions:
S->(L)
S->a
L->Sl
l->,Sl
l->#

```

OUTPUT

```

C:\Users\OSlab\Desktop\gg\1  X  +  v
Enter number of productions: 5
Enter productions:
S->(L)
S->a
L->Sl
l->,Sl
l->#

Grammar After LR Removal:
S->(L)
S->a
L->Sl
l->,Sl
l->#

NT      FIRST      FOLLOW
S       {(a)}      {$,)}
L       {(a)}      {}
l       {,,#}      {}

Parsing Table:
S       (      )      a      ,      $
S->(L)  -      S->a  -      -
L       L->Sl  -      L->Sl  -      -
l       -      l->#  -      l->,Sl  -

-----
Process exited after 4.345 seconds with return value 0
Press any key to continue . . .

```

CONCLUSION

The program successfully converts a given context-free grammar into an LL(1) compatible form by eliminating immediate left recursion, computing accurate FIRST and FOLLOW sets using iterative methods, and constructing the predictive parsing table, thereby demonstrating the complete working of an LL(1) parser in C.

Experiment-8(b)

AIM

Implement the Predictive parsing algorithm, get parse table and input string are inputs. Use C language for implementation.

ALGORITHM

1. Read the number of non-terminals and store them in an array.
2. Read the number of terminals and store them in an array.
3. Input the LL(1) predictive parsing table for all non-terminal and terminal combinations.
4. Display the predictive parsing table in tabular form.
5. Initialize the parsing stack with \$ and the start symbol.
6. Read the input string to be parsed.
7. Compare the stack top with the current input symbol.
8. If both are terminals and match, pop the stack and advance the input pointer.
9. If the stack top is a non-terminal, replace it using the corresponding production from the parse table.
10. Repeat the process until the stack contains \$.
11. Accept the string if both stack top and input symbol are \$, otherwise reject it.

SOURCE CODE

```
#include <stdio.h>
#include <string.h>

#define MAX 50

char NT[20], T[20];
char table[20][20][MAX];
int nNT, nT;

/* ----- Utility ----- */
int isNT(char c) {
    int i;
    for (i = 0; i < nNT; i++)
        if (NT[i] == c) return 1;
    return 0;
}
```

```

int ntIndex(char c) {
    int i;
    for (i = 0; i < nNT; i++)
        if (NT[i] == c) return i;
    return -1;
}

int tlIndex(char c) {
    int i;
    for (i = 0; i < nT; i++)
        if (T[i] == c) return i;
    return -1;
}

/* ----- Print Table ----- */
void printTable() {
    int i, j;
    printf("\nPredictive Parse Table\n");
    for (j = 0; j < nT; j++)
        printf("%c\t", T[j]);
    printf("\n");

    for (i = 0; i < nNT; i++) {
        printf("%c\t", NT[i]);
        for (j = 0; j < nT; j++)
            printf("%s\t", table[i][j]);
        printf("\n");
    }
}

/* ----- Parsing ----- */
void predictiveParse(char *input) {
    char stack[100];
    int top = -1, ip = 0, i;
    char X, a, rhs[MAX];

    stack[++top] = '$';
    stack[++top] = NT[0];

    printf("\nStack\tInput\tAction\n");

    while (top >= 0) {
        for (i = top; i >= 0; i--)
            printf("%c", stack[i]);

        printf("\t%s\t", input + ip);

        X = stack[top];
        a = input[ip];

```



```

    if (!isNT(X)) {
        if (X == a) {
            if (X == '$') {
                printf("Accept\n");
                return;
            }
            printf("Match %c\n", a);
            top--;
            ip++;
        } else {
            printf("Reject\n");
            return;
        }
    } else {
        int r = ntIndex(X);
        int c = tIndex(a);

        if (r == -1 || c == -1 || strcmp(table[r][c], "-") == 0) {
            printf("Reject\n");
            return;
        }

        printf("Output %s\n", table[r][c]);
        top--;
        strcpy(rhs, table[r][c] + 3);

        for (i = strlen(rhs) - 1; i >= 0; i--)
            if (rhs[i] != '#')
                stack[++top] = rhs[i];
    }
}
}

```

```

int main() {
    int i, j;
    char input[50];

    printf("Enter number of non-terminals: ");
    scanf("%d", &nNT);

    printf("Enter non-terminals: ");
    for (i = 0; i < nNT; i++)
        scanf(" %c", &NT[i]);

    printf("Enter number of terminals: ");
    scanf("%d", &nT);

    printf("Enter terminals: ");
    for (i = 0; i < nT; i++)
        scanf(" %c", &T[i]);
}

```

```
printf("Enter parse table:\n");
for (i = 0; i < nNT; i++)
    for (j = 0; j < nT; j++)
        scanf("%s", table[i][j]);

printTable();

printf("Enter input string: ");
scanf("%s", input);

predictiveParse(input);
return 0;
}
```

INPUT

```
3
S L I
5
( ) a , $
S->(L) - S->a - -
L->SI - L->SI - -
- l-># - l->,SI -
(a,(a,a),((a,a)))$
```

OUTPUT

```

C:\Users\OSlab\Desktop\gg\1  X + v
Enter number of non-terminals: 3
Enter non-terminals: S L l
Enter number of terminals: 5
Enter terminals: ( ) a , $
Enter parse table:
S->(L) - S->a - -
L->Sl - L->Sl - -
- l-># - l->,Sl -

Predictive Parse Table
      (      )      a      ,      $
S      S->(L) -      S->a -      -
L      L->Sl -      L->Sl -      -
l      -      l-># -      l->,Sl -
Enter input string: (a,(a,a),((a,a)))$

```

```

Stack   Input   Action
S$      (a,(a,a),((a,a)))$      Output S->(L)
(L)$    (a,(a,a),((a,a)))$      Match (
L)$    a,(a,a),((a,a)))$      Output L->Sl
Sl)$    a,(a,a),((a,a)))$      Output S->a
al)$    a,(a,a),((a,a)))$      Match a
l)$     ,(a,a),((a,a)))$      Output l->,Sl
,Sl)$   ,(a,a),((a,a)))$      Match ,
Sl)$    (a,a),((a,a)))$      Output S->(L)
(Ll)$   (a,a),((a,a)))$      Match (
Ll)$    a,a),((a,a)))$      Output L->Sl
Sl)$    a,a),((a,a)))$      Output S->a
al)$    a,a),((a,a)))$      Match a
ll)$    ,a),((a,a)))$      Output l->,Sl
,Sl)$   ,a),((a,a)))$      Match ,
Sl)$    a),((a,a)))$      Output S->a
al)$    a),((a,a)))$      Match a
ll)$    ),((a,a)))$      Output l->#
)l)$    ),((a,a)))$      Match )
l)$     ,((a,a)))$      Output l->,Sl
,Sl)$   ,((a,a)))$      Match ,
Sl)$    ((a,a)))$      Output S->(L)
(Ll)$   ((a,a)))$      Match (
Ll)$    (a,a)))$      Output L->Sl
Sl)$    (a,a)))$      Output S->(L)
(Ll)l)$ (a,a)))$      Match (
Ll)l)$ a,a)))$      Output L->Sl
Sl)l)l)$ a,a)))$      Output S->a
al)l)l)$ a,a)))$      Match a
l)l)l)$ ,a)))$      Output l->,Sl
,Sl)l)l)$ ,a)))$      Match ,
Sl)l)l)$ a)))$      Output S->a
al)l)l)$ a)))$      Match a
l)l)l)$ )))$      Output l->#
)l)l)$   )))$      Match )
l)l)$    ))$      Output l->#
)l)$     ))$      Match )
l)$      )$      Output l->#
)$       )$      Match )
$        $      Accept

```

CONCLUSION

The program successfully implements an LL(1) predictive parser by using a parsing table and a stack-based approach to verify whether a given input string belongs to the specified grammar, clearly demonstrating the working principle of top-down parsing.

Experiment-9(a) & 9(b)

AIM

- Construct precedence table for the given operator grammar.
- Use the Operator-precedence table in Experiment 9(a), perform the parsing for the given string. (Upload both experiments merged as a single PDF)

ALGORITHM

- Step 1:** Read the grammar (non-terminals, terminals, productions) and the input string.
- Step 2:** Compute the **LEADING** set for each non-terminal using grammar rules.
- Step 3:** Compute the **TRAILING** set for each non-terminal using grammar rules.
- Step 4:** Construct the **operator precedence table** using LEADING and TRAILING sets.
- Step 5:** Initialize stack with \$ and tokenize the input string.
- Step 6:** Apply **operator precedence parsing** using shift and reduce actions based on the precedence table.
- Step 7:** Accept the string if parsing completes successfully, otherwise reject it.

SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

/* --- limits --- */
#define MAX_NT    24
#define MAX_T     30
#define MAX_PROD  64
#define MAX_SYM   16 /* max characters in one symbol */
#define MAX_RHS   40 /* max symbols in one production RHS */
#define MAX_SET   64 /* max size of LEADING / TRAILING set */
#define MAX_TOK   512 /* max tokens in input string */
#define MAX_STK   512 /* max stack depth */
#define MAX_STEPS 999 /* guard against infinite loops */

/* --- relation codes --- */
#define NONE 0
#define LT  1 /* <· shift */
#define GT  2 /* ·> reduce */
#define EQ  3 /* =· shift */

/* --- types --- */
```

```

typedef struct {
    char lhs[MAX_SYM];
    char rhs[MAX_RHS][MAX_SYM];
    int rlen;
} Prod;

/* — globals — */
static char NT [MAX_NT][MAX_SYM]; static int numNT;
static char T  [MAX_T ][MAX_SYM]; static int numT;
static Prod prods[MAX_PROD];      static int numProd;

static int leading [MAX_NT][MAX_SET]; static int leadSz [MAX_NT];
static int trailing[MAX_NT][MAX_SET]; static int trailSz[MAX_NT];

static int prec[MAX_T][MAX_T]; /* prec[i][j] = relation T[i] vs T[j] */

/* — helpers — */
static int ntldx(const char *s){
    int i;
    for(i=0;i<numNT;i++) if(!strcmp(NT[i],s)) return i; return -1;
}
static int tldx(const char *s){
    int i;
    for(i=0;i<numT; i++) if(!strcmp(T[i], s)) return i; return -1;
}
static int isNT(const char *s){ return ntldx(s)>=0; }
static int isT (const char *s){ return tldx (s)>=0; }

static void setAdd(int set[],int *sz,int v){
    int i;
    for(i=0;i<*sz;i++) if(set[i]==v) return;
    set[(*sz)++]=v;
}
static const char *relStr(int r){
    switch(r){case LT:return "<.";case GT:return ">";case EQ:return "=";}
    return " ";
}

/* — LEADING — */
static void computeLeading(void){
    int changed, p, ai, n, bi, j, old;
    char (*rhs)[MAX_SYM];

    memset(leadSz,0,sizeof(leadSz));
    changed=1;
    while(changed){
        changed=0;
        for(p=0;p<numProd;p++){
            ai=ntldx(prods[p].lhs);
            n=prods[p].rlen;

```

```

    rhs=prods[p].rhs;
    if(n==0) continue;
    /* (a) first symbol is terminal */
    if(isT(rhs[0])){
        old=leadSz[ai];
        setAdd(leading[ai],&leadSz[ai],tldx(rhs[0]));
        if(leadSz[ai]!=old) changed=1;
    }
    /* (b) first is NT, second is terminal */
    if(isNT(rhs[0])&&n>=2&&isT(rhs[1])){
        old=leadSz[ai];
        setAdd(leading[ai],&leadSz[ai],tldx(rhs[1]));
        if(leadSz[ai]!=old) changed=1;
    }
    /* (c) first is NT B: propagate LEADING(B) */
    if(isNT(rhs[0])){
        bi=ntldx(rhs[0]);
        for(j=0;j<leadSz[bi];j++){
            old=leadSz[ai];
            setAdd(leading[ai],&leadSz[ai],leading[bi][j]);
            if(leadSz[ai]!=old) changed=1;
        }
    }
}
}
}

/* — TRAILING — */
static void computeTrailing(void){
    int changed, p, ai, n, bi, j, old;
    char (*rhs)[MAX_SYM];

    memset(trailSz,0,sizeof(trailSz));
    changed=1;
    while(changed){
        changed=0;
        for(p=0;p<numProd;p++){
            ai=ntldx(prods[p].lhs);
            n=prods[p].rlen;
            rhs=prods[p].rhs;
            if(n==0) continue;
            /* (a) last symbol is terminal */
            if(isT(rhs[n-1])){
                old=trailSz[ai];
                setAdd(trailing[ai],&trailSz[ai],tldx(rhs[n-1]));
                if(trailSz[ai]!=old) changed=1;
            }
            /* (b) last is NT, second-to-last is terminal */
            if(isNT(rhs[n-1])&&n>=2&&isT(rhs[n-2])){
                old=trailSz[ai];

```

```

        setAdd(trailing[ai],&trailSz[ai],tldx(rhs[n-2]));
        if(trailSz[ai]!=old) changed=1;
    }
    /* (c) last is NT B: propagate TRAILING(B) */
    if(isNT(rhs[n-1])){
        bi=ntldx(rhs[n-1]);
        for(j=0;j<trailSz[bi];j++){
            old=trailSz[ai];
            setAdd(trailing[ai],&trailSz[ai],trailing[bi][j]);
            if(trailSz[ai]!=old) changed=1;
        }
    }
}

/* — Build table ————— */
static void buildTable(void){
    int p, n, k, a, bi, b, dol, si, j;
    char (*rhs)[MAX_SYM];

    memset(prec,NONE,sizeof(prec));
    for(p=0;p<numProd;p++){
        n=prods[p].rlen;
        rhs=prods[p].rhs;
        for(k=0;k<n;k++){
            /* R1a */
            if(isT(rhs[k])&&k+1<n&&isT(rhs[k+1]))
                prec[tldx(rhs[k])][tldx(rhs[k+1])]=EQ;
            /* R1b */
            if(isT(rhs[k])&&k+1<n&&isNT(rhs[k+1])&&k+2<n&&isT(rhs[k+2]))
                prec[tldx(rhs[k])][tldx(rhs[k+2])]=EQ;
            /* R2 */
            if(isT(rhs[k])&&k+1<n&&isNT(rhs[k+1])){
                a=tldx(rhs[k]);
                bi=ntldx(rhs[k+1]);
                for(j=0;j<leadSz[bi];j++){
                    if(prec[a][leading[bi][j]]==NONE)
                        prec[a][leading[bi][j]]=LT;
                }
            }
            /* R3 */
            if(isNT(rhs[k])&&k+1<n&&isT(rhs[k+1])){
                bi=ntldx(rhs[k]);
                b=tldx(rhs[k+1]);
                for(j=0;j<trailSz[bi];j++){
                    if(prec[trailing[bi][j]][b]==NONE)
                        prec[trailing[bi][j]][b]=GT;
                }
            }
        }
    }
}

```

```

/* $ relations */
dol=tldx("$"); if(dol<0) return;
si=0;
for(j=0;j<leadSz[si];j++)
    if(prec[dol][leading[si][j]]==NONE) prec[dol][leading[si][j]]=LT;
for(j=0;j<trailSz[si];j++)
    if(prec[trailing[si][j]][dol]==NONE) prec[trailing[si][j]][dol]=GT;
}

/* — Print sets ————— */
static void printSets(void){
    int i, j;

    printf("|          COMPUTATION OF LEADING SETS          |\n");

    for(i=0;i<numNT;i++){
        printf(" LEADING( %-5s) = { ",NT[i]);
        for(j=0;j<leadSz[i];j++){printf("%-4s",T[leading[i][j]]); if(j<leadSz[i]-1)printf(", ");}
        printf("}\n");
    }
    printf("|          COMPUTATION OF TRAILING SETS          |\n");
    for(i=0;i<numNT;i++){
        printf(" TRAILING( %-5s) = { ",NT[i]);
        for(j=0;j<trailSz[i];j++){printf("%-4s",T[trailing[i][j]]); if(j<trailSz[i]-1)printf(", ");}
        printf("}\n");
    }
}

/* — Print table ————— */
static void printTable(void){
    int i, j;

    printf("| OPERATOR PRECEDENCE TABLE — Experiment 9(a)      |\n");
    printf(" |");
    for(j=0;j<numT;j++) printf(" %-5s|",T[j]);
    printf("\n-----|");
    for(j=0;j<numT;j++) printf("-----|");
    printf("\n");
    for(i=0;i<numT;i++){
        printf(" %-7s|",T[i]);
        for(j=0;j<numT;j++){
            if(prec[i][j]) printf(" %-4s |",relStr(prec[i][j]));
            else          printf("    |");
        }
        printf("\n");
    }
    printf("\n LEGEND: <.=shift(less-than) .>=reduce(greater-than) .=shift(equal) blank=error\n");
}

/* =====

```



```

* PARSING
* ===== */

static int tokenise(const char *inp, char tok[][MAX_SYM], int *n){
    int len, i, best, bestLen, t, tl;
    len=(int)strlen(inp);
    i=0;
    *n=0;
    while(i<len){
        best=-1;
        bestLen=0;
        for(t=0; t<numT; t++){
            tl=(int)strlen(T[t]);
            if(strncmp(inp+i, T[t], tl)==0 && tl>bestLen){best=t; bestLen=tl;}
        }
        if(best>=0){strcpy(tok[(*n)++], T[best]); i+=bestLen;}
        else i++;
    }
    strcpy(tok[(*n)++], "$");
    return *n;
}

static int rightmostTerm(char stk[][MAX_SYM], int top){
    int i;
    for(i=top; i>=0; i--){
        if(isT(stk[i]) && strcmp(stk[i], "$")!=0) return i;
    }
    return -1;
}

static int findHandleLo(char stk[][MAX_SYM], int top, int rt){
    int prev, k, lo;
    prev=-1;
    for(k=rt-1; k>=0; k--){
        if(!strcmp(stk[k], "$")){ prev=k; break; }
        if(isT(stk[k])) { prev=k; break; }
    }
    if(prev<0) return 0;
    lo=prev+1;
    while(lo>0 && lo-1>=0 && isNT(stk[lo-1]) && !strcmp(stk[lo-1], "$")==0){
        if(lo-1==0) break;
        lo--;
    }
    return lo;
}

static int exactMatch(int p, char stk[][MAX_SYM], int lo, int top){
    char st[MAX_RHS][MAX_SYM];
    char pt[MAX_RHS][MAX_SYM];
    int sn, pn, i;

```

```

    sn=0;
    pn=0;
    if(top-lo+1!=prods[p].rlen) return 0;
    for(i=lo;i<=top;i++) if(isT(stk[i])) strcpy(st[sn++],stk[i]);
    for(i=0;i<prods[p].rlen;i++) if(isT(prods[p].rhs[i])) strcpy(pt[pn++],prods[p].rhs[i]);
    if(sn!=pn) return 0;
    for(i=0;i<sn;i++) if(strcmp(st[i],pt[i])!=0) return 0;
    return 1;
}

/* Match by terminal skeleton only (relax length constraint) */
static int skelMatch(int p,char stk[][MAX_SYM],int lo,int top){
    char st[MAX_RHS][MAX_SYM];
    char pt[MAX_RHS][MAX_SYM];
    int sn, pn, i;
    sn=0;
    pn=0;
    for(i=lo;i<=top;i++) if(isT(stk[i])) strcpy(st[sn++],stk[i]);
    for(i=0;i<prods[p].rlen;i++) if(isT(prods[p].rhs[i])) strcpy(pt[pn++],prods[p].rhs[i]);
    if(sn!=pn) return 0;
    for(i=0;i<sn;i++) if(strcmp(st[i],pt[i])!=0) return 0;
    return 1;
}

/* Find matching production; returns index or -1 */
static int findProd(char stk[][MAX_SYM],int lo,int top){
    int p;
    /* 1. Try exact (length + skeleton) */
    for(p=0;p<numProd;p++) if(exactMatch(p,stk,lo,top)) return p;
    /* 2. Try skeleton only (handles cases where the NT in handle was already replaced) */
    for(p=0;p<numProd;p++) if(skelMatch(p,stk,lo,top)) return p;
    return -1;
}

static void mkStkStr(char stk[][MAX_SYM],int top,char *out){
    int i;
    out[0]='\0';
    for(i=0;i<=top;i++){strcat(out,stk[i]);if(i<top)strcat(out," ");}
}

static void mkInpStr(char tok[][MAX_SYM],int ip,int n,char *out){
    int j;
    out[0]='\0';
    for(j=ip;j<n;j++){strcat(out,tok[j]);if(j<n-1)strcat(out," ");}
}

static void parseInput(const char *inputStr){
    char tok[MAX_TOK][MAX_SYM];
    char stk[MAX_STK][MAX_SYM];
    char ss[700], is[400], act[256];
    int numTok, top, ip, steps, accepted, err;

```

```
int rt, a, b, rel, lo, pi, wlo, k;
```

```
numTok=0;  
tokenise(inputStr,tok,&numTok);
```

```
top=-1;  
strcpy(stk[++top],"$");  
ip=0;
```

```
printf("|      OPERATOR PRECEDENCE PARSING      |\n");  
printf("| Input string : %-78s|\n",inputStr);
```

```
printf(" %-40s | %-25s | %-5s | %s\n","Stack","Remaining Input","Rel","Action");  
printf(" %-40s-+-%-25s-+-%-5s-+-%-40s\n",  
    "-----",  
    "-----",  
    "----",  
    "-----");
```

```
steps=0;  
accepted=0;  
err=0;  
while(!accepted&&!err){  
    mkStkStr(stk,top,ss);  
    mkInpStr(tok,ip,numTok,is);
```

```
/* — ACCEPT CHECK ————— */  
if(!strcmp(tok[ip],"$")){  
    /* Accept: stack = [$ <start-NT>] */  
    if(top==1&&!strcmp(stk[0],"$")&&isNT(stk[1])){  
        printf(" %-40s | %-25s | %-5s | ACCEPT\n",ss,is,"");  
        accepted=1; break;  
    }  
    /* Accept: stack = [$ <start-NT> $] (some parsers push $) */  
    if(top==2&&!strcmp(stk[0],"$")&&isNT(stk[1])&&!strcmp(stk[2],"$")){  
        printf(" %-40s | %-25s | %-5s | ACCEPT\n",ss,is,"=.");  
        accepted=1; break;  
    }  
}
```

```
/* — Find rightmost non-$ terminal on stack ————— */  
rt=rightmostTerm(stk,top);
```

```
if(rt<0){  
    /* No real terminal: just shift */  
    if(!strcmp(tok[ip],"$")){  
        printf(" %-40s | %-25s | %-5s | ACCEPT\n",ss,is,"");  
        accepted=1; break;  
    }  
}
```

```

    sprintf(act,"SHIFT  %s",tok[ip]);
    printf(" %-40s | %-25s | %-5s | %s\n",ss,is,"",act);
    strcpy(stk[++top],tok[ip++]);
    continue;
}

a=tldx(stk[rt]);
b=tldx(tok[ip]);
if(a<0||b<0){
    printf(" %-40s | %-25s | %-5s | ERROR: unknown symbol\n",ss,is,"");
    err=1; break;
}

/* Special: $ (as topmost terminal) vs $ on input → accept */
if(!strcmp(stk[rt],"$")&&!strcmp(tok[ip],"$")){
    printf(" %-40s | %-25s | %-5s | ACCEPT\n",ss,is,".");
    accepted=1; break;
}

rel=prec[a][b];

if(rel==LT||rel==EQ){
    /* SHIFT */
    sprintf(act,"SHIFT  %s",tok[ip]);
    printf(" %-40s | %-25s | %-5s | %s\n",ss,is,relStr(rel),act);
    strcpy(stk[++top],tok[ip++]);
}
else if(rel==GT){
    /* REDUCE */
    lo=findHandleLo(stk,top,rt);

    /* Try to find production with progressively wider windows */
    pi=-1;
    for(wlo=lo; wlo>=0&&pi<0; wlo--){
        /* try stk[wlo..top] */
        pi=findProd(stk,wlo,top);
        if(pi>=0) { lo=wlo; break; }
        if(wlo==0) break;
    }

    if(pi>=0){
        sprintf(act,"REDUCE by  %s -> ",prods[pi].lhs);
        for(k=0;k<prods[pi].rlen;k++) strcat(act,prods[pi].rhs[k]);
        printf(" %-40s | %-25s | %-5s | %s\n",ss,is,relStr(rel),act);
        /* Pop the handle (lo..top) and push LHS */
        top=lo-1;
        strcpy(stk[++top],prods[pi].lhs);
    } else {
        printf(" %-40s | %-25s | %-5s | ERROR: no matching production for
handle\n",ss,is,relStr(rel));
    }
}

```

```

        err=1;
    }
}
else {
    sprintf(act,"ERROR: prec['%s']['%s'] undefined",T[a],T[b]);
    printf("  %-40s | %-25s | %-5s | %s\n",ss,is,"",act);
    err=1;
}

if(++steps>MAX_STEPS){ printf(" ERROR: %d steps exceeded.\n",MAX_STEPS); err=1; }
}

printf("\n");
if(accepted)
    printf(" ==> Input string  \"%s\" is ACCEPTED. <==\n",inputStr);
else
    printf(" ==> Input string  \"%s\" is REJECTED. <==\n",inputStr);
}

static void trimLine(char *s){
    int n=(int)strlen(s);
    while(n>0&&(s[n-1]=='\n' || s[n-1]=='\r' || s[n-1]==' ')) s[--n]='\0';
}

static int splitByComma(const char *src,char arr[][MAX_SYM]){
    int cnt, tlen, dup, i;
    const char *p, *q, *s, *e;
    char tok[MAX_SYM];

    cnt=0;
    p=src;
    while(1){
        q=p;
        while(*q&&*q!=' ') q++;
        /* trim spaces */
        s=p; while(s<q&&*s==' ') s++;
        e=q-1; while(e>=s&&*e==' ') e--;
        tlen=(int)(e-s+1);
        if(tlen<=0) strcpy(tok,""); /* empty field = literal comma */
        else { strncpy(tok,s,tlen); tok[tlen]='\0'; }
        /* deduplicate */
        dup=0;
        for(i=0;i<cnt;i++) if(!strcmp(arr[i],tok)){dup=1;break;}
        if(!dup) strcpy(arr[cnt++],tok);
        if(!*q) break;
        p=q+1;
    }
    return cnt;
}

```

```

static void parseProdLine(const char *line, Prod *prod){
    char buf[256];
    char *arrow, *lhs, *le, *rhs;
    int pos, rlen, best, bestLen, bestType, i, tl;

    strncpy(buf, line, 255); buf[255] = '\0'; trimLine(buf);
    arrow = strstr(buf, "->"); if(!arrow){prod->rlen=-1;return;}
    *arrow = '\0';
    lhs = buf; while(*lhs == ' ') lhs++;
    le = lhs + strlen(lhs) - 1; while(le > lhs && *le == ' ') *le-- = '\0';
    strcpy(prod->lhs, lhs);
    rhs = arrow + 2;
    prod->rlen = 0;
    pos = 0;
    rlen = (int)strlen(rhs);
    while(pos < rlen){
        best = -1; bestLen = 0; bestType = 0;
        for(i = 0; i < numNT; i++){
            tl = (int)strlen(NT[i]);
            if(strncmp(rhs + pos, NT[i], tl) == 0 && tl > bestLen){best = i; bestLen = tl; bestType = 1;}
        }
        for(i = 0; i < numT; i++){
            tl = (int)strlen(T[i]);
            if(strncmp(rhs + pos, T[i], tl) == 0 && tl > bestLen){best = i; bestLen = tl; bestType = 2;}
        }
        if(bestLen > 0){
            if(bestType == 1) strcpy(prod->rhs[prod->rlen++], NT[best]);
            else strcpy(prod->rhs[prod->rlen++], T[best]);
            pos += bestLen;
        } else pos++;
    }
}

int main(void){
    char line[512];
    char inputStr[256];
    char tmp[512];
    char *t;
    int i, k;

    printf("|   OPERATOR PRECEDENCE PARSER   |\n");

    printf("Enter grammar and input (format described in comments at top):\n\n");

    /* 1. Number of NTs */
    if(!fgets(line, sizeof(line), stdin)) return 1; trimLine(line);
    numNT = atoi(line);

```

```

/* 2. NT symbols */
if(!fgets(line,sizeof(line),stdin)) return 1; trimLine(line);
if((int)strlen(line)==numNT&&!strchr(line',')){
    for(i=0;i<numNT;i++){NT[i][0]=line[i];NT[i][1]='\0';}
} else if(strchr(line',')){
    numNT=splitByComma(line,NT);
} else {
    numNT=0; strcpy(tmp,line);
    t= strtok(tmp, " ");
    while(t&&numNT<MAX_NT){strcpy(NT[numNT++],t);t= strtok(NULL, " ");}
}

/* 3. Number of terminals */
if(!fgets(line,sizeof(line),stdin)) return 1; trimLine(line);
numT=atoi(line);

/* 4. Terminal symbols (comma-separated; empty = comma terminal) */
if(!fgets(line,sizeof(line),stdin)) return 1; trimLine(line);
numT=splitByComma(line,T);

/* 5. Number of productions */
if(!fgets(line,sizeof(line),stdin)) return 1; trimLine(line);
numProd=atoi(line);

/* 6. Productions */
for(i=0;i<numProd;i++){
    if(!fgets(line,sizeof(line),stdin)) break;
    trimLine(line);
    parseProdLine(line,&prods[i]);
}

/* 7. Input string */
if(!fgets(line,sizeof(line),stdin)) return 1; trimLine(line);
strcpy(inputStr,line);

/* — Echo grammar — */
printf("|          GRAMMAR SUMMARY          |\n");
printf(" Non-terminals : ");
for(i=0;i<numNT;i++) printf("[%s] ",NT[i]); printf("\n");
printf(" Terminals   : ");
for(i=0;i<numT; i++) printf("[%s] ",T[i]); printf("\n");
printf(" Productions : \n");
for(i=0;i<numProd;i++){
    printf("  P%-2d %s -> ",i+1,prods[i].lhs);
    for(k=0;k<prods[i].rlen;k++) printf("%s ",prods[i].rhs[k]);
    printf("\n");
}
printf(" Input string : %s\n",inputStr);

/* — Compute, display, parse — */

```

```
computeLeading();  
computeTrailing();  
printSets();  
buildTable();  
printTable();  
parseInput(inputStr);  
  
printf("\n");  
return 0;  
}
```

INPUT

```
2  
SL  
5  
a,,,($  
4  
S->(L)  
S->a  
L->L,S  
L->S  
(a,(a,a))
```

OUTPUT

Enter grammar and input (format described in comments at top):

```

Number of Non-terminals
Non-terminals no space
Number of Terminals eg. a,b,c,d
Number of Productions
Productions
Input string
2
SL
5
a,,,(),$
4
S->(L)
S->a
L->L,S
L->S
(a,(a,a))
|
GRAMMAR SUMMARY
Non-terminals : [S] [L]
Terminals : [a] [,] [(] [)] [$]
Productions :
P1 S -> ( L )
P2 S -> a
P3 L -> L , S
P4 L -> S
Input string : (a,(a,a))
|
COMPUTATION OF LEADING SETS
LEADING( S ) = { ( , a }
LEADING( L ) = { , ( , a }
|
COMPUTATION OF TRAILING SETS
TRAILING( S ) = { ) , a }
TRAILING( L ) = { , , ) , a }

```

C:\Users\thanu\Downloads\U x + v

OPERATOR PRECEDENCE TABLE ù Experiment 9(a)					
	a	,	(	)	\$
a		.>		.>	.>
,	<.	.>	<.	.>	
(	<.	<.	<.	=.	
)		.>		.>	.>
\$	<.		<.		

LEGEND: <.=shift(less-than) .>=reduce(greater-than) .=shift(equal) blank=error

OPERATOR PRECEDENCE PARSING ù Experiment 9(b)

Input string : (a,(a,a))

Stack	Remaining Input	Rel	Action
\$	(a , (a , a)) \$		SHIFT (
\$(	a , (a , a)) \$	<.	SHIFT a
\$(a	, (a , a)) \$	.>	REDUCE by S → a
\$(S	, (a , a)) \$	<.	SHIFT ,
\$(S ,	(a , a)) \$	<.	SHIFT (
\$(S , (	a , a)) \$	<.	SHIFT a
\$(S , (a	, a)) \$	.>	REDUCE by S → a
\$(S , (S	, a)) \$	<.	SHIFT ,
\$(S , (S ,	a)) \$	<.	SHIFT a
\$(S , (S , a	)) \$	.>	REDUCE by S → a
\$(S , (S , S	)) \$	.>	REDUCE by L → L,S
\$(S , (L	)) \$	=.	SHIFT)
\$(S , (L)	) \$	.>	REDUCE by S → (L)
\$(S , S	) \$	.>	REDUCE by L → L,S
\$(L	) \$	=.	SHIFT)
\$(L)	\$	.>	REDUCE by S → (L)
\$ S	\$		ACCEPT

====> Input string "(a,(a,a))" is ACCEPTED. <====

CONCLUSION

Thus, the given input string is successfully analyzed using an Operator Precedence Parser by computing LEADING and TRAILING sets, constructing the precedence table, and performing shift-reduce parsing to accept or reject the string.